

CH2 机器学习与前馈神经网络

- 1. 前馈神经网络里面，它有一些什么样的方法？
信息仅在一个方向上流动，从输入层通过隐藏层最终到达输出层，信息传递过程中，没有反馈循环。
- 2. 前馈神经网络它怎么去训练？
前向传播，计算预测值；损失计算：做叉熵、均方误差；
反向传播：计算梯度；参数更新：使用梯度下降法；
- 3. 权重和偏置，权重是什么样的？偏置是什么样的？
权重表征连接强度，它控制了输入特征对输出结果的影响力大小。
偏置是一个向量，核心作用是提供灵活性，平移决策边界。
- 4. 要知道什么叫做正向传播，什么叫做误差的反向传播？

对第 $l-1$ 层前向传播，生成预测值 $\hat{y} = x^l$ ：
 $x^l = ReLU(\mathbf{k}x^{l-1} + \mathbf{b})$ ；其中，类似于 $ReLU(\cdot)$ 的激活函数，是神经网络中非常重要的组成部分，它向网络引入非线性特性，使网络能够学习复杂的函数；

反向传播：是通过链式法则，从网络的输出层开始，逐层向输入层（反向）计算损失函数对于每一个权重和偏置的梯度；为后边使用优化算法，迭代模型参数做好准备；

- 5. 如何选择损失函数？
面向回归任务，我们预测的是连续值，采用均方误差。
对异常值敏感；曲线特性良好，易于优化；
面向分类任务，我们预测的是类别标签，采用交叉熵损失。
其中，二元分类配合 Sigmoid 激活，使用交叉熵损失；多元分类配合 Softmax 激活，使用分类交叉熵损失；

任务类型	输出层激活函数	损失函数	公式&说明
多分类 (图像分类)	Softmax (输出概率分布)	交叉熵损失	$L = -\sum_{c=1}^M y_{o,c} \log(p_{o,c})$ M: 类别数, y: 真实标签 (one-hot), p: 预测概率
二分类 (真/假)	Sigmoid (输出 0~1 概率值)	二元交叉熵	$L = -[y \log(p) + (1-y) \log(1-p)]$ y: 真实标签(0 或 1), p: 预测为 1 的概率

回归

(预测坐标)

ReLU

(直接输出实数值)

均方误差

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

y : 真实值, $\hat{y}$: 预测值

多标签分类

(一张图多个
标签)

多个 Sigmoid

(每个类别独立判断)

多标签交叉熵

将二元交叉熵损失对每个标
签求和或平均

6. 如何选择优化函数?

看效率和效果: 首选 Adam 算法, 追求极致泛化或作为基线时可尝试 SGD(with Momentum)。

7. 怎么让你的这个学习率去下降?

学习率衰减, 是指随着训练, 逐步减小学习率 lr 。

可以采用: 余弦退火; 阶梯衰减; 或者使用自适应优化器 (如 Adam) 自动调整;

8. 这个随机梯度是一个什么样的方法?

【定义】每次只随机抽取一个样本。用这个样本计算的“随机”度来近似真实梯度, 并立即更新参数;

【特点】内存效率高, 收敛速度快, 能够跳出局部最优, 适用于大规模数据;

9. 你怎么去做网络的优化?

权重和偏置的优化是一个统一、自动化的过程, 统称为“优化模型参数”。这个过程的核心是梯度下降算法及其变种。

通过计算损失函数相对于每个参数的梯度, 然后沿着梯度反方向 (即下降最快的方向) 更新参数, 以最小化损失。

10. 怎么去减少这个泛化的错误? 为什么要去做这个正则化?

引入正则化, 实现结构风险最小化, 达到简化模型或者抑制模型的过拟合;

其中, ℓ_1 正则化稀疏权重, 用来简化模型; ℓ_2 正则化平滑权重, 用来防止过拟合;

11. 你要通过一些什么样的方法, 去做这个正则化的改进?

Dropout

训练时随机“关闭”一部分神经元, 强制网络不依赖特定路径;

提前停止 Early Stop

使用验证集监控性能, 当验证集错误率不再下降时停止训练;

批归一化 Batch Normalize

标准化每层输入, 加速训练, 提高稳定性

数据增强

对训练数据进行变换 (如旋转、裁剪), 增加数据多样性

12. 什么叫做经验风险最小化, 什么叫做结构风险最小化?

经验风险最小化: 目标是最小化模型在训练集上的平均损失。它只关心“拟合数据”, 但可能导致过拟合。

结构风险最小化：在经验风险的基础上，加上一个正则化项 来惩罚模型复杂度。

目标是同时“拟合数据”和“保持模型简单”，从而追求更好的泛化能力。

13. 前馈神经网络它的网络架构是什么？FNN 它有什么样的层，它每一层做了什么？

不考虑 Dropout 的情形下，前馈神经网络是一种分层结构，信息从输入层到输出层单向传递，层与层之间全连接，层内无连接，整个网络构成一个有向无环图。

FNN 包含有输入层，隐藏层，输出层；其中，输入层负责接收原始特征，隐藏层用于特征提取，输出层预测结果；

14. 激活函数它对应的一些下游任务是什么？

激活函数	公式	特点	适用场景
Sigmoid	$\sigma(x)=\frac{1}{1+e^{-x}}$	输出(0,1), 梯度易消失	输出层: 二元分类
Tanh	$\tanh(x)=\frac{e^x-e^{-x}}{e^x+e^{-x}}$	输出(-1,1), 零中心, 梯度易消失	隐藏层: RNN/LSTM
ReLU	$\text{ReLU}(x)=\max(0,x)$	计算快, 解决 部分梯度消失, 但可能"死亡"	隐藏层: 默认 CNN/FNN
Leaky ReLU	$\text{LeakyReLU}(x)=\max(\alpha x, x)$	解决"Dying ReLU"问题	隐藏层: ReLU 的改进版
Softmax	$\text{Softmax}(z_i)=\frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$	输出概率分布, 和为 1	输出层: 多元分类

15. FNN 的优势以及 FNN 所对应的问题？

结构简单，易于理解和实现；
适合结构化数据的分类、回归、评分等任务；
是复杂神经网络（CNN, RNN, Transformer）的基础模块；

无法建模空间/时序依赖，如图像、文本序列等复杂数据；
层数过深容易出现梯度消失/爆炸，难以训练；
难以自动提取高级特征；

CH3 卷积神经网络

1. 卷积层是什么？怎么实现卷积操作？

卷积层是卷积神经网络中的核心组件，用于自动提取输入数据的局部特征。它的设计灵感来源于生物学中的感受野机制；采用局部连接和权重共享的方式，减少权值的数量和减低模型复杂度；

卷积操作本质上是一个可移动的窗口（卷积核）在输入数据上滑动，进行逐元素相乘后求和的过程。

2. 感受野是什么？感受野怎么去做计算？

感受野是指卷积神经网络每一层输出的特征图上的一个像素点，所对应输入图像上的区域大小。感受野的计算由下式给出： $RF_i = (RF_{i+1} - 1) \times stride_i + Kernel$ ；最后一层感受野的大小是最后一层卷积核的大小；

3. 卷积神经网络里面的局部连接和权重共享，它们之间是一个什么样的关系呢？是怎么去结合这两个，去优化你的卷积神经网络？

局部连接解决了“怎么看？”的问题，规定了每个神经元只感受输入的一小片局部区域；权重共享解决了“用什么看”的问题，规定了在一个通道内，在图像的不同位置，使用同一个卷积核进行处理。

极大减少参数量，防止过拟合；保留空间结构，并具有平移不变性；实现高效的特征提取；显著提升计算效率；

4. 在 CNN 网络中，一张图片经过卷积核为 3×3 ，步长为 2 的卷积层 Conv，ReLU 激活函数，Batch Normalize 层，以及一个步长为 2 卷积核为 2×2 的池化层 Pooling 后，再经过一 3×3 的卷积层，步长为 1，此时的感受野为多少？

二级结论，最后一层感受野大小 3×3 ；公式做题，倒数第二 Pooling 层 $(3-1) \times 2 + 2 = 6$ ；公式做题，倒数第三 Conv 层 $(6-1) \times 2 + 3 = 13$ ；感受野的大小是 13×13 ；

5. 神经网络的典型结构，以及 CNN 中每个模块的具体用途是什么？

一个典型的 CNN 结构序列如下：输入层 $\rightarrow$ [卷积层 $\rightarrow$ 激活函数 $\rightarrow$ 池化层] $\times N$ $\rightarrow$ 全连接层 $\rightarrow$ 输出层；

其中，

输入层负责接收和预处理原始数据，对输入图像做：尺寸调整，归一化，去均值；

卷积层负责自动提取输入数据的局部特征；

池化层对特征图进行降维，保持主要特征的同时增加感受野，并引入一定程度的不变性。

全连接层将卷积层和池化层学习到的分布式特征表示映射到样本标记空间。

CH4 循环神经网络

1. 在 RNN 网络中，他的核心创新点？

循环神经网络（RNN）最重要的核心创新点在于引入了“[带自反馈的循环连接](#)”，这使得网络具有了记忆能力，能够处理序列数据并捕捉时间上的依赖关系。

循环神经网络中节点或层的输出不仅传递给下一层，同时也作为输入反馈给自身（或同层的其他节点）。

2. 循环神经网络输入输出的计算？假设输入序列是二维向量，偏置为 0，激活函数是线性 $f(x) = x$ ；初始隐藏状态 $h_0 = [0,0]$ ；其中

$$x_1 = [1,1], x_2 = [1,1], x_3 = [2,2]$$

$$W_{xh} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, W_{hh} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, W_{hy} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

展开来算就是了：

$$h_1 = W_{xh}X_1 + W_{hh}h_0 = [2,2]; \quad y_1 = W_{hy} h_1 = [4,4];$$

$$h_2 = W_{xh}X_2 + W_{hh}h_1 = [6,6]; \quad y_2 = W_{hy} h_2 = [12,12];$$

$$h_3 = W_{xh}X_3 + W_{hh}h_2 = [16,16]; \quad y_3 = W_{hy} h_3 = [32,32];$$

3. 循环神经网络的计算单元在哪里？

输入层：负责接收输入序列 x_t 。

隐藏层：是带有循环连接的层，负责进行核心计算和维持“记忆状态”。其过程可

以表示为： $S_t = f(U \cdot X_t + W \cdot S_{t-1} + b)$

输出层：根据隐藏层的状态生成输出 y_t

4. 什么是图灵完备？

所有的图灵机都可以被一个由使用 Sigmoid 型激活函数的神经元构成的全连接循环网络来进行模拟。

5. 解释同步&异步的序列到序列模型

同步的序列到序列模式输入输出等长，并行计算速度快，计算效率高；经典任务包括：分词、NER、词性标注、语音识别；

异步的序列到序列模式，输入输出不等长，顺序生成，能捕捉输出序列内部的依赖关系；但处理速度慢，无法并行计算整个输出序列；经典任务包括：机器翻译、文本摘要、对话生成、代码生成；

6. 循环神经网络里面怎么去做这个参数的更新，然后怎么去做这个梯度的下降？

不要重新琢磨，这个在“抽象层”的流程，和前馈神经网络基本上一模一样。但是注意到：RNN 损失是**所有时间步损失的总和**；误差不仅反向传播通过网络层，还**反向传播通过时间**；梯度计算涉及在**所有时间步上求和的链式法则**。

7. 循环神经网络存在梯度消失、梯度爆炸的问题，基于循环神经网络本身，有哪些方法可以去改变？

当时间步 $t - k$ 很大时，即存在长程依赖），我们需要连续乘以很多个矩阵。如果这个矩阵的范数 > 1 ，梯度会指数级增长 → 梯度爆炸；如果这个矩阵的范数 < 1 ，梯度会指数级衰减 → 梯度消失。

为了解决梯度爆炸问题，可以使用梯度截断、权重衰减方法，直接从数值上限制梯度的大小或权重的规模。

为了解决梯度消失问题，引入了 LSTM、GRU 去提升处理长序列文本的能力；

8. 长短期记忆网络它的怎么去做循环的，它有一些什么样的特点？

细胞状态是 LSTM 的“主干线”，负责在序列中长期保存信息。它在时间步之间水平传递，只进行线性操作（如加法、乘法），变化缓慢，旨在解决梯度消失问题。

隐藏状态是 LSTM 的“输出状态”，是由细胞状态衍生而来的，包含了当前时间步的相关信息，会传递给输出层和下一个时间步。

下一个时间步的“记忆” (C_t) 和“输出” (h_t) 共同由当前输入 x_t 、上一时间的记忆 C_{t-1} 和上一时间的输出 h_{t-1} 决定。

遗忘门决定从过去的细胞状态 C_{t-1} 中丢弃哪些信息得到；输入门，将确定更新的部分 i_t 和候选值 $\hat{C}_t$ ；基于当前的细胞状态 C_t ，决定下一个隐藏状态 h_t 应该是什么；

CH5 网络的优化与正则化

好消息！考点只有一个~~

如果我们网络存在一些“具体”的问题，理解这些问题并提出针对性的解决方案。

1. 方法论 概述：

优化算法：如 SGD、Momentum、Adam 等，用于加速收敛、逃离局部最优。

学习率策略：如衰减、周期性调整、自适应学习率，用于平衡收敛速度与稳定性。

结构设计：如残差连接、归一化层、激活函数选择，用于改善优化地形。

正则化技术：如 Dropout、权重衰减、数据增强，用于防止过拟合。

初始化与归一化：如 He/Xavier 初始化、Batch Norm/Layer Norm，用于稳定训练过程。

2. 具体问题对应关系

问题类别	具体问题	解决方法	说明
结构差异大	没有通用优化算法、超参数多	• 更有效的优化算法 • 超参数优化方法（网格搜索、随机搜索、贝叶斯优化）	不同网络结构需针对性选择优化策略
非凸优化问题	鞍点、局部最优、平坦/尖锐最小值	• 动量法（Momentum） • 自适应学习率算法（Adam, RMSprop） • 周期性学习率（Cyclical LR） • Nesterov 加速梯度	避免陷入鞍点，寻找平坦最小值以提高泛化能力
梯度消失/爆炸	深层网络中梯度不稳定	• 更好的参数初始化（Xavier, He） • 逐层归一化（Batch Norm, Layer Norm） • 残差连接（ResNet） • 梯度截断 • 使用 ReLU/Leaky ReLU	保持前向/反向传播中信号稳定

训练不稳定	学习率不合适、梯度噪声大	• 学习率衰减（阶梯、线性、余弦） • 小批量梯度下降（Mini-batch） • 梯度方向优化（Momentum, Adam） • 自适应学习率（Adagrad, RMSprop, Adadelta）	平衡收敛速度与稳定性
过拟合/泛化差	模型复杂、训练集好测试集差	• 正则化（L1/L2, Dropout, 权重衰减） • 早停（Early Stopping） • 数据增强 • 标签平滑	控制模型复杂度，提高泛化能力
参数初始化问题	对称权重、梯度消失/爆炸	• Xavier 初始化（Tanh/Sigmoid） • He 初始化（ReLU） • 正交初始化（RNN/LSTM）	避免初始梯度不稳定
数据分布不稳定	内部协变量偏移	• 批量归一化（BN） • 层归一化（LN） • 实例归一化（IN） • 组归一化（GN）	稳定每层输入分布，加速训练

CH6 注意力机制

1. 阐述注意力打分机制？

加性模型 $s(x, q) = v^T \tanh(Wx + Uq)$ ；参数量较多，达到 $d_h \times (d_x + d_q + 1)$ ；表达能力较强；

点积模型 $s(x, q) = x^T q$ ；计算简单高效，无参数量；x 与 q 要求维度相同；

缩放点积模型 $s(x, q) = \frac{x^T q}{\sqrt{D}}$ ；计算简单高效，无参数量；不能学习复杂的匹配函数，灵活性低。

双加性模型 $s(x, q) = x^T W q$ ；参数量大；可以学习复杂的交互关系，灵活性高；适用于需要精细交互的任务；

2. 关注注意力的模型，处理这个不同的下游任务的时候，怎么去做这个注意力？

注意力类型	主要特点	适用任务
软性注意力	加权平均所有输入	通用
硬性注意力	只选一个输入	图像分类、指针网络
键值对注意力	分离匹配与内容	检索、阅读理解
多头注意力	多视角并行注意力	Transformer、机器翻译
交叉注意力	两个序列相互关注	NLI、阅读理解
自注意力	序列内部全局交互	文本表示、预训练模型

3. 关注 Transformer 结构，并做主要的解释：

每一层编码器包含两个核心子层：

多头自注意力层：让序列中的每一个词都能够关注到序列中所有其他词，从而捕捉全局的上下文信息。每个词生成 Query, Key, Value 向量，通过计算注意力权重，得到所有词的加权和作为输出。

前馈神经网络层：对每个位置的表示进行非线性变换和增强。

残差连接与层归一化：在每个子层（自注意力层、前馈层）周围，都有一个残差连接，随后进行层归一化。可以缓解深层网络中的梯度消失问题，稳定训练过程。

每一层解码器包含三个核心子层：

带掩码的多头自注意力层：确保在预测第 i 个位置时，只能看到 1 到 $i-1$ 位置的信息，防止信息泄露，这是自回归生成的关键。

多头编码器-解码器注意力层：让解码器在生成当前词时，能够聚焦于编码器输出的源序列中最相关的部分。其中，Query 来自解码器上一层的输出，Key 和 Value 来自编码器的最终输出。

4. 有关于记忆网络的事情

方法	核心思想	记忆操作	主要特点
记忆网络	构建外部记忆库，用注意力查询	主要为读	结构简单，常用于问答
神经图灵机	模仿图灵机，可微分读写	读和写	开创性，引入了可微分的注意力寻址
可微分神经计算机	NTM 的增强，复杂内存管理	读和写	具有内存分配和时序链接，推理能力更强
联想记忆网络	内容寻址，能量最小化	动态检索	生物启发，擅长模式补全

CH7 深度生成模型

1. 生成对抗网络的主体架构，每个模块都在那边做什么？

生成器（Generator）

输入：随机噪声向量 $z \sim p(z)$ （通常来自高斯分布）；

输出：生成的数据样本 $G(z)$ ，如图像；

目标：生成逼真的数据，使得判别器无法区分其与真实数据；

判别器（Discriminator）

输入：真实数据 x 或生成数据 $G(z)$

输出：一个标量，表示输入是真实数据的概率

目标：尽可能准确地区分真实数据与生成数据

2. VAE 变分自编码器 VS GAN 生成对抗网络

VAE 的本质是一个带概率约束的自编码器。它的目标不是直接对抗，而是学习数据的概率分布。VAE 没有真正的对抗，但它通过一个巧妙的损失函数来实现两个互相“拉扯”的目标：其中，重建损失希望解码器输出的 x' 和原始输入 x 越像越好，让编码器输出有意义的潜变量；KL 散度损失希望编码器输出的分布 $q(z|x)$ 尽量接近标准正态分布 $p(z) = N(0, I)$ ，规范潜空间，防止编码器过拟合；

GAN 的核心思想是博弈论中的零和博弈，直接体现出了“对抗”过程。先固定生成器 G ，训练判别器 D ，目标是最大化判别器的判断能力。再固定判别器 D ，训练生成器 G ，目标是最小化生成器被识破的概率。

理想状态下，当生成器强大到判别器完全无法分辨真假（时，训练达到平衡。此时，生成器产生的数据分布与真实数据分布几乎重合。

方面	VAE（变分自编码器）	GAN（生成对抗网络）
核心理想	概率建模，学习一个规整的潜空间	对抗博弈，直接学习数据分布
“对抗”本质	损失函数内部的对抗： 重建损失 vs KL 散度损失	模型之间的对抗： 生成器 vs 判别器
训练方式	联合训练，同时优化编码器和解码器	交替训练，轮流固定一个、训练另一个
优点	训练稳定，潜空间具有解释性，天生能做编码	生成质量通常更高，图像更清晰、锐利

3. DCGAN 它有一些什么样的架构，DCGAN 做了一些什么样的创新？

全卷积网络：用带步长的卷积来实现下采样操作，不用最大池化（Maxpooling），让网络自己学习下采样的方式。

取消全连接层：使用全局平均池化代替全连接层，降低模型复杂度，全局平均池化可以提高模型稳定性。

批归一化：只对生成器和鉴别器的输入层使用，可以加速收敛和减缓过拟合，否则会导致样本震荡和模型不稳定。

LeakyReLU 激活函数：使用 LeakyReLU 作为激活函数，解决了传统 ReLU 在生成器中可能导致的梯度稀疏问题，有助于提高训练稳定性。

4. 前向 KL 散度 & 反向 KL 散度

前向 KL 散度 $D_{KL}(P \parallel Q)$ ，是避免 Q 忽略了 P 的任何模式，即：在真实数据容易出现的地方， p_θ 是否也分配了足够的概率密度；

反向 KL 散度 $D_{KL}(Q \parallel P)$ ，是避免 Q 在 P 为零处“无中生有”，即：在模型生成的数据范围内，这些数据是否与真实数据高度吻合；

5. 你怎么去改进可能在训练干网络的时候出现的梯度消失 梯度爆炸 网络崩塌的这个问题？

梯度消失是指：在最优判别器下，生成器的优化目标等价于最小化 P_{data} 和 P_g 之间的 JS 散度。当两个分布没有重叠时，JS 散度为常数，梯度为 0。

于是，改进生成器的**损失函数**（不最小化 $\log(1 - D(G(z)))$ ），而是改为**最大化 $\log(D(G(z)))$ 或最小化 $-\log(D(G(z)))$** ；但是，坏消息，出现了**模式崩塌**。

模式崩塌：一边既要**最小化 $D_{KL}(p_\theta \parallel p_r)$ （逆向 KL 散度）**，然后又要**最大化 $D_{JS}(p_r \parallel p_\theta)$ （JS 散度）**；

改进方法是：用 **Wasserstein 距离（推土机距离）** 代替 JS 散度；

附录：

梯度爆炸问题(提到了，PPT 没细说)：

改进方法是：梯度惩罚；上 Adam/RMSProp 自适应优化器；学习率调整；

6. WGAN，他做了一些什么样的改进，它的优势是什么？

彻底抛弃了判别器的概率输出，改用 **Wasserstein 距离**。即使在没有分布重叠的情况下也能提供平滑且有意义的梯度，从根本上解决了梯度消失问题。**W** 距离鼓励生成器覆盖全部真实分布，极大缓解了模式崩塌问题。